

0. Importing External Libraries

Reference PyPi.org for installation, importing, and documentation

```
# importing a library w/o alias
import tifffile
```

```
# importing a library w/ alias
import tifffile as tiff
```

```
# importing portion of a library
import matplotlib.pyplot
from matplotlib import pyplot
```

```
# combine portion + alias
import matplotlib.pyplot as plt
```

External libraries we use in this lesson

Library/Module	Description	Import Syntax
tifffile	Reads & writes tiff files	<code>import tifffile</code>
ndv	View images interactively	<code>import ndv</code>
matplotlib.pyplot	Module within matplotlib for plotting	<code>import matplotlib.pyplot</code>
numpy	N-dim array computing	<code>import numpy</code>
bffile	Reads proprietary file types	<code>import bffile</code>

1. Reading .tif Image w/ tifffile

Use `tifffile` to read .tif images and store as `numpy` array

```
import tifffile as tiff
image_path = r"/path/to/image.tif"
image = tiff.imread(image_path)
```

2. Image Viewing w/ ndv

View images in an interactive viewer

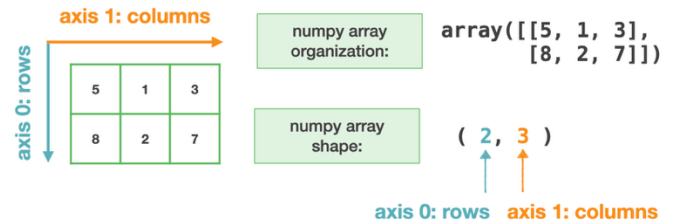
```
import ndv
ndv.imshow(image)
```

3. Image Viewing w/ matplotlib

View images statically as a plot

```
import matplotlib.pyplot as plt
plt.imshow(image)
```

4. numpy Arrays: 2D Images



Working with 2D numpy arrays

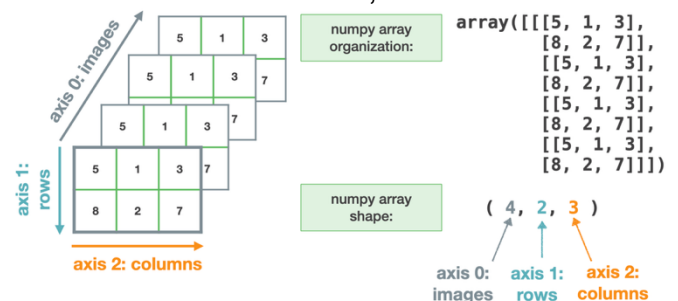
Task	Syntax
Find shape	<code>image.shape</code>
Find bit depth (dtype)	<code>image.dtype</code>
Confirm type	<code>type(image)</code>
Access individual pixel values with indices	<code>image[0,0]</code>
Access an entire row of pixel values w/indices	<code>image[0,:]</code> or <code>image[0]</code>
Access an entire column of pixel values w/indices	<code>image[:,0]</code>

Accessing pixel statistics across the array

Statistic	Syntax
Minimum	<code>image.min()</code>
Maximum	<code>image.max()</code>
Mean	<code>image.mean()</code>

5. numpy Arrays: Multi-D Images

Examples of multi-dimensional images: z-stacks, time lapses, multiple fluorescence channels, etc.



Working with multi-d numpy arrays

Task	Syntax
Find shape	<code>image.shape</code>
Find bit depth (dtype)	<code>image.dtype</code>
Confirm type	<code>type(image)</code>
Access individual images from 3D array w/indices	<code>image[0, :, :]</code> or <code>image[0]</code>

Image projections

Projection Type	Syntax
Max Intensity Projection	<code>numpy.max(image, axis=0)</code>
Min Intensity Projection	<code>numpy.min(image, axis=0)</code>
Sum Projection	<code>numpy.sum(image, axis=0)</code>
Avg Intensity Projection	<code>numpy.mean(image, axis=0)</code>
Stdev Projection	<code>numpy.std(image, axis=0)</code>
Variance Projection	<code>numpy.var(image, axis=0)</code>

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 2,
                        figsize=(8,6))
axes[0].imshow(img_1)
axes[1].imshow(img_2)
plt.show()
```

Miscellaneous plot settings

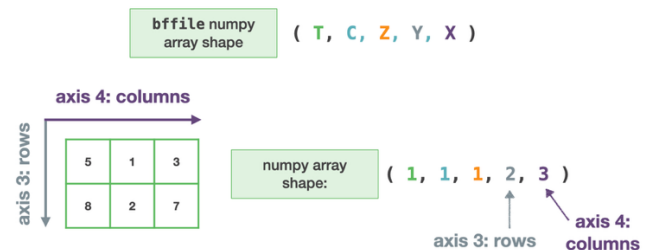
Setting	Syntax
Remove axis labels	<code>axes[0].axis("off")</code>
Changing colormap	<code>axes[0].imshow(img_1, cmap="gray")</code>
Adjusting B&C	<code>axes[0].imshow(img_1, vmin=img_1.min(), vmax=img_1.max())</code>
Fitting subplots in the figure	<code>fig, axes = plt.subplots(1, 2, figsize=(8,6), layout="constrained")</code>

8. Reading Proprietary File Types w/ bfile

Use **bfile** to read proprietary image file types (e.g., .nd2, .czi, .lif, etc)

```
import bfile
image_path = "/path/to/image.czi"
image = bfile.imread(image_path)
```

how **bfile** organizes a 2D image as a numpy array



Use **numpy.squeeze()** to drop axes with length 1 from a **numpy** array

Strategy	Syntax
Drop a single axis with length 1	<code>np.squeeze(image, axis = 0)</code>
Drop any axes with length 1	<code>np.squeeze(image)</code>

6. Image Saving w/ tiff file

Use **tiff file** to save .tif images

```
import tiff file as tiff
output_path =
    "/path/to/image_name.tif"
tiff.imwrite(output_path,
            image.dtype = "uint8")
```

7. Plotting Multiple Images

```
fig, axes = plt.subplots(1, 2, figsize=(8,6))
```

